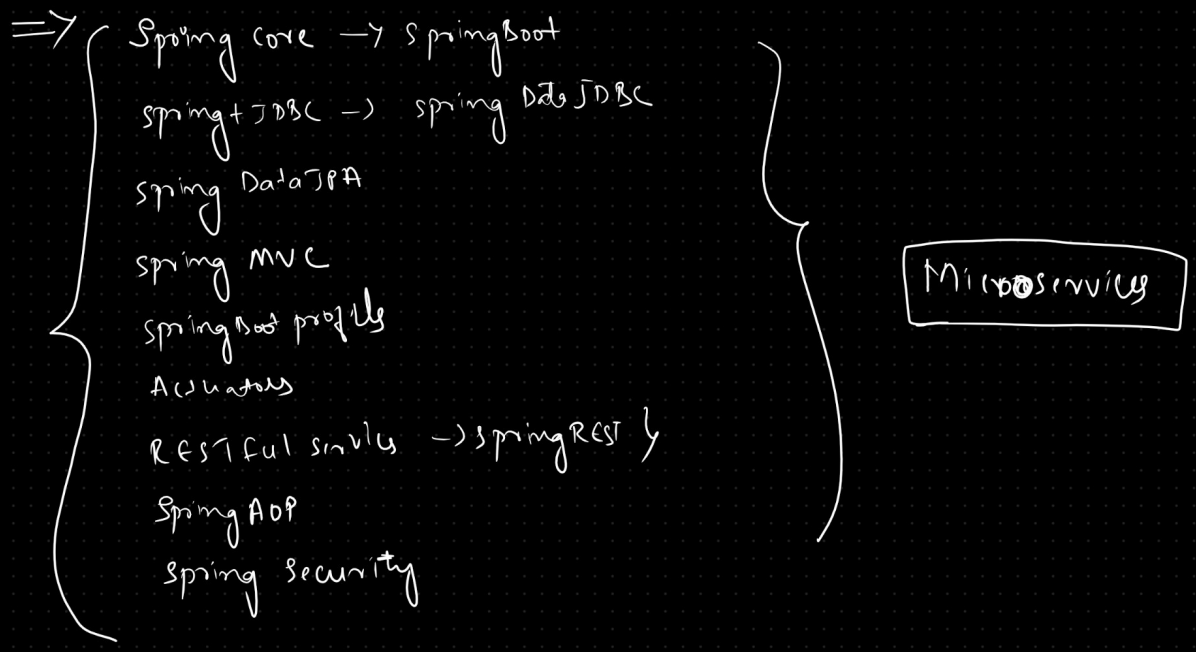
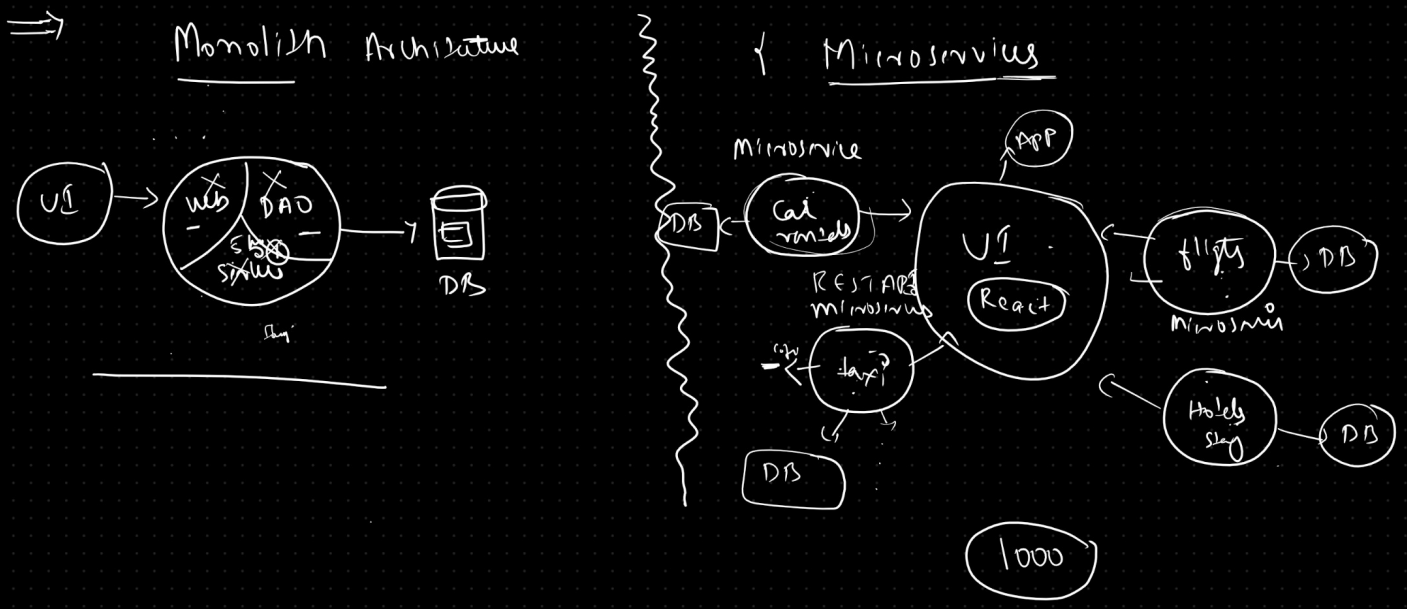




Monolith Architecture:

-->A monolith application includes all functionalities in a single project.
-->It's packaged as a jar/war file and deployed to a server.
-->Since it contains everything, it results in a large jar/war file.

Advantages of Monolith:

->Simple to develop.
->Everything is in one place.
->Only one configuration is needed.

Disadvantages of Monolith:

->Hard to maintain.
->Single point of failure.
->Requires redeploying the entire project for updates.

To resolve the above issues we use Microservices Architecture.

What are Microservices?

->Microservices is an architectural design pattern.
(Microservices is not a programming language, framework, or API)

->It suggests developing application functionalities as loosely coupled components.

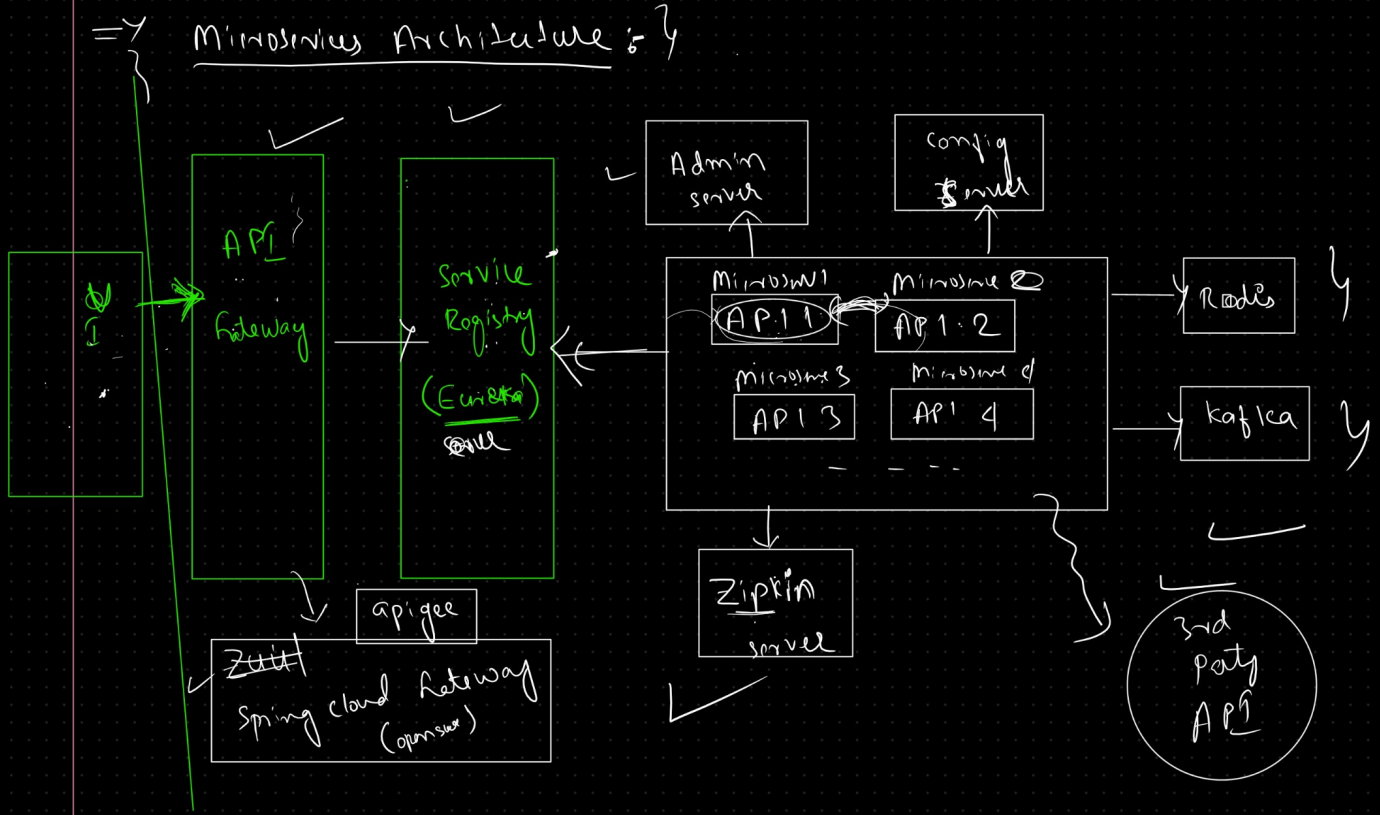
->Instead of one big project, functionalities are divided into multiple REST APIs, each called a microservice.

Advantages of Microservices:

->Loosely coupled components.
->Easier to maintain.
->Faster development.
->Quick deployment.
->Faster releases.
->Less downtime.
->Technology independence (different technologies can be used for different services).

Disadvantages of Microservices:

->Deciding the number of services to create (bounded context).
->Requires a lot of configurations.
->Harder to see the overall system (visibility).



Microservices Architecture:

->Common components in Microservices Architecture include:

Service Registry (Eureka Server): Database of available services.
(Services (REST APIs or Microservice): Individual microservices containing backend business logic.)

API Gateway: Manages and routes requests to backend APIs.

Admin Server: Manages backend API endpoints.

Zipkin Server: For distributed tracing to monitor API performance.

Config server

Apache Kafka

Rediscache

Service Registry
->Acts as a database for the services in the project.
->Lists all registered services and their instances.
->Eureka Server from the "Spring Cloud Netflix" library is commonly used.

API Gateway
->Manages backend APIs and acts as a mediator between users and backend APIs.
->Includes filters for request processing (e.g., authentication) and routing logic.
>Spring Cloud Gateway can be used as an API Gateway.

Admin Server
->Manages all backend API actuator endpoints in one place.
->Provides a user interface to monitor API endpoints.

Zipkin Server
->Used for distributed tracing to see which API takes the most time to process requests.

Apache Kafka
A distributed messaging system used for building real-time data pipelines and streaming applications.
Facilitates communication between microservices by publishing and subscribing to streams of records (messages).

Redis Cache
An in-memory data structure store used as a database, cache, and message broker.
Speeds up the retrieval of frequently accessed data, reducing the load on databases and improving application performance.